

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

За седьмой семестр

По дисциплине: «Современные методы защиты
компьютерных систем»

Тема: Разработка модулей ядра операционной системы для динамического
перехвата системных вызовов с использованием технологии eBPF

Выполнил:

Студент 4 курса
Группы ИИ-26(2).
Рулько М.А.

Проверил:

Хацкевич А.С.

Брест 2026

Цель работы:

практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Задание 1. Базовый перехват запуска процессов

1. Разработайте eBPF-зонд (C + Python), использующий механизм перехвата kprobe (или tracepoint) для отслеживания системного вызова `execve` (запуск новых процессов).
2. Напишите логику на стороне ядра (C), которая извлекает идентификатор процесса (PID), идентификатор пользователя (UID) и имя исполняемого файла (`comm`).
3. Запустите скрипт с правами `root` и продемонстрируйте перехват запуска стандартных утилит (например, `ls` или `cat`) в соседнем окне терминала или новой вкладке WSL.

Наш основной файл `main.py`

`check_root()` — проверяет что скрипт запущен от `root`. Если нет — выводит ошибку и завершает работу. eBPF требует привилегий суперпользователя.

`load_c(filename)` — читает C-файл зонда из папки `ebpf/` и возвращает его содержимое как строку. BCC принимает C-код именно в виде строки и компилирует его под текущее ядро на лету.

`run_task1()` — загружает зонд `task1_execve.c` в ядро, привязывает Python-callback к кольцевому буферу `execve_events` и запускает бесконечный цикл опроса буфера. При каждом запуске любого процесса в системе ядро кладёт событие в буфер, Python его читает и выводит в терминал.

`run_task2()` — аналогично загружает `task2_kill.c`, но буфер `kill_events` получает события только когда C-код в ядре пропустил фильтр `target_pid == 1`. Python выводит алерт в терминал и записывает строку в `security_audit.log`.

```

1. import argparse
2. import os
3. import sys
4.
5. from bcc import BPF
6.
7. import monitor
8.
9. def check_root() -> None:
10.     if os.geteuid() != 0:
11.         print("[ERROR] Скрипт должен быть запущен с правами root: sudo python3 main.py")
12.         sys.exit(1)
13.
14. def load_c(filename: str) -> str:
15.     path = os.path.join(os.path.dirname(__file__), "ebpf", filename)
16.     with open(path, "r") as f:
17.         return f.read()
18.
19. def run_task1() -> None:
20.     print("[*] Задание 1: мониторинг запуска процессов (execve)")
21.     print("[*] Загружаем eBPF-зонд в ядро...")
22.
23.     bpf = BPF(text=load_c("task1_execve.c"))
24.     bpf["execve_events"].open_perf_buffer(monitor.handle_execve_event)
25.
26.     print("[*] Зонд активен. Запускайте команды в соседнем терминале (ls, cat, ...)")
27.     print("[*] Для остановки нажмите Ctrl+C\n")
28.     print(f"{'-'*60}")
29.     print(f" {'Время':<20} {'Тип':<8} {'Детали'}")
30.     print(f"{'-'*60}")
31.
32.     try:
33.         while True:
34.             bpf.perf_buffer_poll()
35.     except KeyboardInterrupt:
36.         print("\n[*] Мониторинг остановлен.")
37.
38. def run_task2() -> None:
39.     print("[*] Задание 2+3: детектор попытки kill(pid=1) + аудит лога")
40.     print("[*] Загружаем eBPF-зонд в ядро...")
41.
42.     bpf = BPF(text=load_c("task2_kill.c"))
43.
44.     bpf["kill_events"].open_perf_buffer(monitor.handle_kill_event)
45.
46.     print("[*] Зонд активен. Для генерации аномалии выполните в другом терминале:")
47.     print("    kill -9 1")
48.     print("    или:")
49.     print("    python3 -c \"import os, signal; os.kill(1, signal.SIGTERM)\"")
50.     print(f"[*] Алерты также пишутся в: {os.path.abspath(monitor.LOG_FILE)}")
51.     print("[*] Для остановки нажмите Ctrl+C\n")
52.     print(f"{'!'*60}")
53.     print(f" Ожидаем аномалию: kill(target_pid=1, ...)")
54.     print(f"{'!'*60}")
55.
56.     try:
57.         while True:
58.             bpf.perf_buffer_poll()
59.     except KeyboardInterrupt:
60.         print("\n[*] Мониторинг остановлен.")
61.
62. def parse_args() -> argparse.Namespace:
63.     parser = argparse.ArgumentParser(
64.         description="eBPF-монитор. Лабораторная работа №3, вариант 2.",
65.         formatter_class=argparse.RawTextHelpFormatter,
66.     )
67.     parser.add_argument(
68.         "--task",
69.         type=int,
70.         choices=[1, 2],
71.         required=True,
72.         help=(
73.             "Номер задания:\n"
74.             "  1 – перехват execve (запуск процессов)\n"
75.             "  2 – детектор kill(pid=1) + запись в security_audit.log"
76.         ),

```

```

77.     )
78.     return parser.parse_args()
79.
80. def main() -> None:
81.     check_root()
82.     args = parse_args()
83.
84.     if args.task == 1:
85.         run_task1()
86.     elif args.task == 2:
87.         run_task2()
88.
89. if __name__ == "__main__":
90.     main()
91.

```

task1_execve.c

```

1. #include <uapi/linux/ptrace.h>
2. #include <linux/sched.h>
3.
4. /* Структура события, которую передаём в Python через кольцевой буфер */
5. struct execve_event_t
6. {
7.     u32 pid;
8.     u32 uid;
9.     char comm[TASK_COMM_LEN]; /* имя процесса, макс. 16 символов */
10. };
11.
12. /* Объявляем кольцевой буфер для передачи событий в user space */
13. BPF_PERF_OUTPUT(execve_events);
14.
15. /*
16.  * Хук на tracepoint sys_enter_execve.
17.  * Срабатывает при каждом запуске нового процесса.
18.  */
19. TRACEPOINT_PROBE(syscalls, sys_enter_execve)
20. {
21.     struct execve_event_t event = {};
22.
23.     /* Получаем PID и UID текущего процесса */
24.     event.pid = bpf_get_current_pid_tgid() >> 32;
25.     event.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
26.
27.     /* Копируем имя процесса из пространства ядра */
28.     bpf_get_current_comm(&event.comm, sizeof(event.comm));
29.
30.     /* Отправляем событие в кольцевой буфер */
31.     execve_events.perf_submit(args, &event, sizeof(event));
32.
33.     return 0;
34. }
35.

```

Вывод нашей команды в **WSL** после ввод ls + cat
к сожалению я вводил команды через powershell – так что название
команды переопределяется bash либо relay!

```
[*] Зонд активен. Запускайте команды в соседнем терминале (ls, cat, ...)
[*] Для остановки нажмите Ctrl+C
```

Время	Тип	Детали			
[2026-10-07 12:19:55]	EXECVE	PID=785	UID=0	COMM=Relay	
[2026-10-07 12:19:55]	EXECVE	PID=785	UID=0	COMM=bash	
[2026-10-07 12:19:59]	EXECVE	PID=789	UID=0	COMM=Relay	
[2026-10-07 12:19:59]	EXECVE	PID=789	UID=0	COMM=bash	
[2026-10-07 12:20:41]	EXECVE	PID=792	UID=0	COMM=Relay	
[2026-10-07 12:20:41]	EXECVE	PID=792	UID=0	COMM=bash	
[2026-10-07 12:20:46]	EXECVE	PID=795	UID=0	COMM=Relay	
[2026-10-07 12:20:46]	EXECVE	PID=795	UID=0	COMM=bash	
[2026-10-07 12:20:49]	EXECVE	PID=798	UID=0	COMM=Relay	
[2026-10-07 12:20:49]	EXECVE	PID=798	UID=0	COMM=bash	
^C					
[*] Мониторинг остановлен.					

Задание 2. Динамический анализ аномального поведения

1. Определите целевой системный вызов и условие аномального поведения согласно вашему индивидуальному варианту.
2. Напишите eBPF-зонд на C, извлекающий аргументы выбранного системного вызова (например, путь к целевому файлу, целевой PID или сетевой порт).
3. Реализуйте логику фильтрации строго на стороне ядра: программа должна передавать событие в пространство пользователя (через кольцевой буфер BPF_PERF_OUTPUT) *только* при совпадении с условием аномалии. Передача всех подряд событий в Python-скрипт является грубой ошибкой производительности.
4. Сгенерируйте целевую аномалию (например, попытайтесь прочитать защищенный файл) и зафиксируйте успешный перехват алерта вашим скриптом.

task2_kill.c

```
1. #include <uapi/linux/ptrace.h>
2. #include <linux/sched.h>
3.
4. /* Структура события для передачи в Python */
5. struct kill_event_t
6. {
7.     u32 attacker_pid;          /* PID процесса, вызвавшего kill() */
8.     u32 attacker_uid;          /* UID пользователя-нарушителя */
9.     u32 target_pid;            /* целевой PID (всегда 1 в наших алертах) */
10.    int signal;                 /* номер сигнала (9 = SIGKILL, 15 = SIGTERM, ...) */
11.    char comm[TASK_COMM_LEN]; /* имя процесса-нарушителя */
12. };
13.
14. /* Объявляем кольцевой буфер */
15. BPF_PERF_OUTPUT(kill_events);
16.
17. /*
18.  * Хук на tracepoint sys_enter_kill.
19.  * args->pid - первый аргумент kill(): целевой PID
20.  * args->sig - второй аргумент kill(): сигнал
21.  */
22. TRACEPOINT_PROBE(syscalls, sys_enter_kill)
23. {
24.
25.     /* ——— ФИЛЬТР ЯДРА — начало ———
26.      * Если целевой PID не равен 1 — немедленно выходим.
27.      * В буфер ничего не пишем, Python ничего не получает.
28.      * Это и есть фильтрация на стороне ядра.
29.      */
30.     if (args->pid != 1)
31.     {
32.         return 0;
33.     }
34.     /* ——— ФИЛЬТР ЯДРА — конец ——— */
35.
36.     /* Дошли сюда только если target_pid == 1 — это аномалия */
37.     struct kill_event_t event = {};
38.
39.     event.attacker_pid = bpf_get_current_pid_tgid() >> 32;
40.     event.attacker_uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
41.     event.target_pid = args->pid;
42.     event.signal = args->sig;
43.
44.     bpf_get_current_comm(&event.comm, sizeof(event.comm));
45.
46.     /* Отправляем алерт в кольцевой буфер */
47.     kill_events.perf_submit(args, &event, sizeof(event));
48.
49.     return 0;
50. }
```

Вывод нашей программы

При каждом вызове `kill()` в системе срабатывает наш код.

Только если `target_pid == 1` — собирает структуру события с PID нарушителя, его UID, именем процесса и номером сигнала, и отправляет в кольцевой буфер `kill_events`.

```
10 warnings generated.
[*] Зонд активен. Для генерации аномалии выполните в другом терминале:
    kill -9 1
или:
    python3 -c "import os, signal; os.kill(1, signal.SIGTERM)"
[*] Алерты также пишутся в: /root/lab3/security_audit.log
[*] Для остановки нажмите Ctrl+C

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Ожидаем аномалию: kill(target_pid=1, ...)
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[ALERT] ПОПЫТКА УБИТЬ PID 1 (systemd/init)
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Timestamp   : 2026-10-07 12:23:06
Process     : bash (PID=820)
UID         : 0
Target PID  : 1
Signal      : 9
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

[LOG] Запись добавлена в /root/lab3/security_audit.log
```

Задание 3. Интеграция с пространством пользователя и аудит

1. Расширьте Python-скрипт из задания 2: при получении события об аномалии из ядра скрипт должен автоматически записывать предупреждение в локальный лог-файл `security_audit.log`.
2. Формат записи должен содержать: временную метку (Timestamp), PID нарушителя, UID нарушителя и детали события.

Monitor.py

```
1. import ctypes
2. import datetime
3. import os
4.
5. _IN_DOCKER = os.path.exists("/.dockerenv")
6. LOG_FILE = "/app/logs/security_audit.log" if _IN_DOCKER else "security_audit.log"
7.
8. def _write_log(line: str) -> None:
9.     """Дописывает одну строку в файл security_audit.log."""
10.     with open(LOG_FILE, "a") as f:
11.         f.write(line + "\n")
12.
13. def _now() -> str:
14.     return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
15.
16. class ExecveEvent(ctypes.Structure):
17.     _fields_ = [
18.         ("pid", ctypes.c_uint32),
19.         ("uid", ctypes.c_uint32),
20.         ("comm", ctypes.c_char * 16),
21.     ]
22.
23. def handle_execve_event(cpu, data, size):
24.     event = ctypes.cast(data, ctypes.POINTER(ExecveEvent)).contents
25.     comm = event.comm.decode("utf-8", errors="replace")
26.     ts = _now()
27.     print(f"[{ts}] EXECVE | PID={event.pid:<7} UID={event.uid:<5} COMM={comm}")
28.
29. class KillEvent(ctypes.Structure):
30.     _fields_ = [
31.         ("attacker_pid", ctypes.c_uint32),
32.         ("attacker_uid", ctypes.c_uint32),
33.         ("target_pid", ctypes.c_uint32),
34.         ("signal", ctypes.c_int32),
35.         ("comm", ctypes.c_char * 16),
36.     ]
37.
38. def handle_kill_event(cpu, data, size):
39.     event = ctypes.cast(data, ctypes.POINTER(KillEvent)).contents
40.     comm = event.comm.decode("utf-8", errors="replace")
41.     ts = _now()
42.
43.     alert = (
44.         f"\n{'!'*60}\n"
45.         f" [ALERT] ПОПЫТКА УБИТЬ PID 1 (systemd/init)\n"
46.         f"{'!'*60}\n"
47.         f" Timestamp : {ts}\n"
48.         f" Process : {comm} (PID={event.attacker_pid})\n"
49.         f" UID : {event.attacker_uid}\n"
50.         f" Target PID : {event.target_pid}\n"
51.         f" Signal : {event.signal}\n"
52.         f"{'!'*60}\n"
53.     )
54.     print(alert)
55.
56.     log_line = (
57.         f"[{ts}] "
58.         f"ALERT | "
59.         f"PID={event.attacker_pid} | "
60.         f"UID={event.attacker_uid} | "
61.         f"COMM={comm} | "
62.         f"EVENT=kill(target_pid={event.target_pid}, sig={event.signal})"
63.     )
64.     _write_log(log_line)
65.     print(f" [LOG] Запись добавлена в {os.path.abspath(LOG_FILE)}\n")
66.
```

Вывод нашей программы — так как находимся в WSL, то файл откроем прямо в консоли.

```
root@DESKTOP-7C4I8KV:~/lab3# cat ~/lab3/security_audit.log
[2026-10-07 12:23:06] ALERT | PID=820 | UID=0 | COMM=bash | EVENT=kill(target_pid=1, sig=9)
root@DESKTOP-7C4I8KV:~/lab3#
```

Задание 4

Seccomp-BPF vs eBPF kprobes

Seccomp-BPF — механизм блокировки. Он стоит перед ядром и перехватывает `syscall` до его выполнения. Если правило говорит "запретить `kill()`" — процесс получит ошибку и вызов не выполнится вообще. Именно на этом основаны песочницы браузеров и контейнеры.

eBPF kprobe — механизм наблюдения. Зонд срабатывает когда `syscall` уже выполняется внутри ядра. Остановить его оттуда нельзя — ядро уже в процессе работы, его структуры данных частично изменены. Попытка вмешаться может привести к `kernel panic`. Поэтому kprobe только смотрит и сообщает, но не блокирует.

Когда нужна именно блокировка — используют Seccomp-BPF или LSM-хуки, которые специально встроены в точки принятия решений ядра.

TOCTOU применительно к зонду

TOCTOU — это атака на временной разрыв между проверкой и использованием данных.

Простой пример: атакующий пишет программу с двумя потоками. Первый поток вызывает `kill(1, 9)`. В момент входа в `syscall` наш зонд читает аргумент и видит `pid=1` — фиксирует аномалию. Но второй поток в этот же момент подменяет аргумент в памяти на безобидный `pid=1234`. Ядро выполняет уже подменённый вызов. Итог: зонд сработал ложно, реальной атаки не было — или наоборот, атака прошла незамеченной.

Защита — читать аргументы не из памяти процесса, а из регистров процессора (`pt_regs`), куда ядро их скопировало при входе в `syscall`. Эту копию атакующий изменить уже не может. Именно так и работает наш зонд через `bpf_probe_read`.

Вывод:

В ходе лабораторной работы были освоены основные принципы разработки eBPF-программ для мониторинга событий уровня ядра Linux. Был реализован практический стенд на базе WSL2 с использованием фреймворка BCC.